
Namespaces & Multi-Tenancy in Kubernetes

Namespaces, combined with quotas and limits, form the foundation for **multi-tenancy** in Kubernetes — allowing multiple teams or applications to share a single cluster while maintaining **isolation**, **fair resource usage**, and **access control**.

1. Namespace

What it is:

A **logical partition** of cluster resources that enables the separation of concerns, environments, or tenants.

Why use it:

- Isolate applications (e.g., dev, staging, prod)
- Apply RBAC policies per team or project
- Use quotas and policies independently for each namespace

Key characteristics:

- Resource names must be unique **within a namespace**, but **can repeat across namespaces**
- Some resources are **not namespaced**, like **nodes** and **persistentvolumes**

Example:

```
kubectl create namespace dev-team-a  
kubectl get pods -n dev-team-a
```

2. ResourceQuota

What it is:

Defines **maximum resource consumption limits** for a namespace, such as:

- CPU
- Memory
- Number of pods
- Number of services, secrets, etc.

Purpose:

- Prevent a single namespace from exhausting cluster resources
- Enforce fairness and prevent abuse in multi-tenant clusters

Enforced per namespace.

Example:

```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: dev-quota
  namespace: dev
spec:
  hard:
    pods: "20"
    requests.cpu: "4"
    requests.memory: "8Gi"
    limits.cpu: "10"
    limits.memory: "16Gi"
```

If a pod tries to request more than these limits or causes the namespace to exceed them, it's rejected.

3. LimitRange

What it is:

Specifies **default and maximum/minimum resource requests and limits** per container or pod in a namespace.

Why it's useful:

- Ensures all pods have resource limits (avoids "unbounded" pods)
- Applies **default values** when omitted
- Helps the scheduler make informed decisions

Enforced per namespace.

Example:

```
apiVersion: v1
kind: LimitRange
metadata:
  name: container-limits
  namespace: dev
spec:
  limits:
    - default:
        cpu: 500m
        memory: 512Mi
      defaultRequest:
        cpu: 250m
        memory: 256Mi
    type: Container
```

Explanation:

- If a user creates a container without specifying CPU/memory, the defaults are applied.
 - Users cannot request more than the defined limits unless explicitly overridden.
-

Combined Example: Namespace with Quota and LimitRange

```
kubectl create namespace team-a
```

```
# ResourceQuota for team-a
```

```
apiVersion: v1
```

```
kind: ResourceQuota
```

```
metadata:
```

```
  name: team-a-quota
```

```
  namespace: team-a
```

```
spec:
```

```
  hard:
```

```
    pods: "10"
```

```
    requests.cpu: "2"
```

```
    limits.cpu: "4"
```

```
# LimitRange for team-a
```

```
apiVersion: v1
```

```
kind: LimitRange
```

```
metadata:
```

```
  name: team-a-defaults
```

```
  namespace: team-a
```

```
spec:
```

```
  limits:
```

```
    - type: Container
```

```
      default:
```

```
        cpu: 300m
```

```
        memory: 256Mi
```

```
      defaultRequest:
```

```
        cpu: 150m
```

memory: 128Mi

Namespaces and Security / Access Control

Namespaces also integrate with:

- **RBAC (Role-Based Access Control)** to restrict who can perform actions in a namespace.
 - **NetworkPolicies** to restrict cross-namespace traffic.
 - **ServiceAccounts** and policies to isolate workloads by tenant or team.
-

Summary Table

Concept	Purpose	Scope
Namespace	Logical isolation of resources and permissions	Cluster-wide
ResourceQuota	Caps total usage (CPU, memory, object count)	Per namespace
LimitRange	Sets default/request/limit per container	Per namespace
